

Перенос среды между провайдерами

Как забрать всё, что человек настроил под один агентский CLI — правила, скиллы, команды, субагентов, хуки, права, MCP-серверы, переменные окружения и ключи — и получить ту же работу на любом другом. Что переезжает один в один, что доигрывает панель, а что невозможно и почему.

Версия документа: 18.09.2026 · Проектный документ: описывает, как это должно быть устроено. План работ с задачами и критериями приёмки — `TASKS-PORTABILITY.md` в корне репозитория. Факты о форматах десяти CLI взяты из адаптеров панели, уже написанных и покрытых тестами.

Что внутри

- 1. Задача: что именно теряется при переходе
- 2. Решение: канон вместо девяноста переводов
- 3. Канон несёт смысл, а не байты
- 4. Пять уровней верности
- 5. Матрица: слой × провайдер
- 6. Транспилиция по слоям: было → стало
- 7. Надзиратель рантайма: панель как оболочка чужого CLI
- 8. Провод: то, чего не видно снаружи процесса
- 9. Два режима владения: перенос и подписка
- 10. Контекст незакрытой работы
- 11. Честная граница: что работает без панели
- 12. Чем это доказывается

1. Задача: что именно теряется при переходе

«Перейти с одного CLI на другой» звучит как замена одной программы другой. На деле среда человека — это не программа, а десяток разных механизмов, и работает она так, как он привык, только когда все они на месте одновременно.

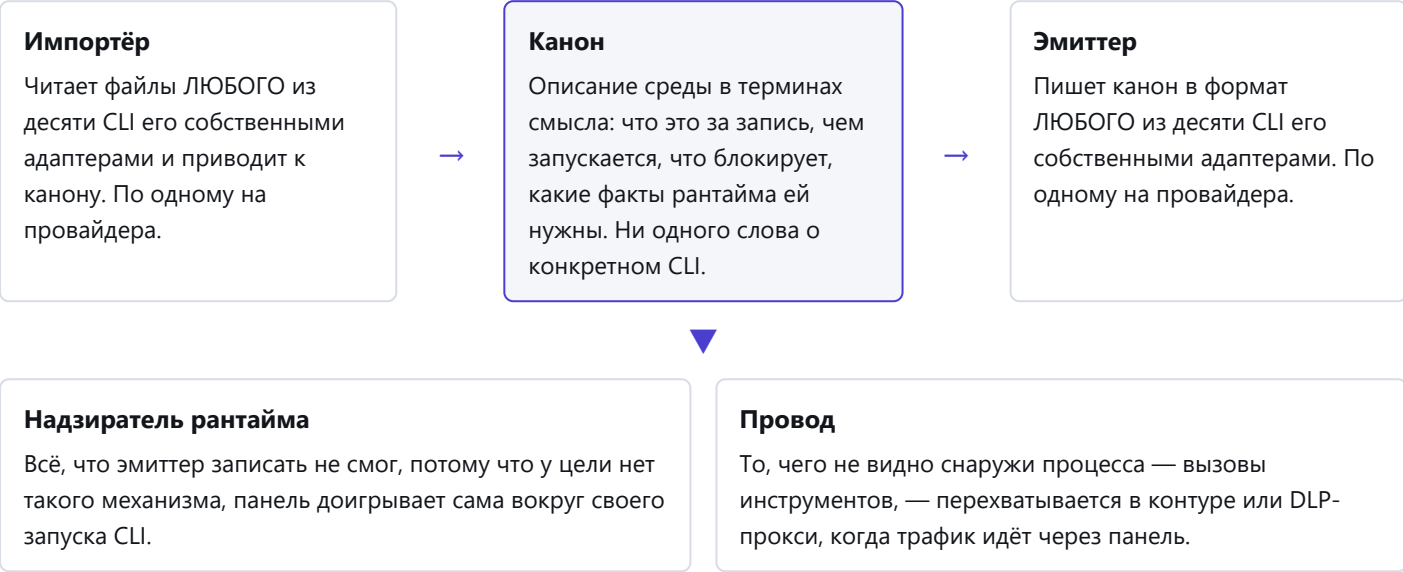
слой	что это у CLAUDE CODE	что происходит сегодня при переходе
Инструкции и память	<code>CLAUDE.md</code> , его <code>@</code> -импорты, разделы «ПРАВИЛО:»	Копируется текстом — единственное, что сегодня переносится
Скиллы	<code>skills/<имя>/SKILL.md</code> + раскрытие по мере надобности	Не переносится. У части CLI механизма нет вовсе
Слэш-команды	<code>commands/**/*.*.md</code> , плюс команды плагинов	Не переносится

Субагенты	<code>agents/*.md</code> — отдельные роли со своим набором инструментов	Не переносится. Аналога нет ни у кого
Хуки	9 событий, скрипт на событие, код выхода 2 блокирует действие	Не переносится. У <code>codex</code> , <code>gemini</code> и ещё четырёх хуков нет вовсе
Права	Правила <code>allow/ask/deny</code> с грамматикой вида <code>Bash(git push:*)</code>	Панель отказывается переносить прямым текстом: «у CLI разные модели согласований»
MCP-серверы	<code>mcpServers</code> в <code>~/.claude.json</code>	Переносится — второе и последнее, что умеет сегодняшний перенос
Переменные и ключи	<code>settings.json</code> → <code>env</code> , секреты в зашифрованном хранилище панели	Панель отказывается: «в них хранятся ключи»
Группы и сценарии	Наборы сущностей и автоматизации панели	Живут в панели, но компилируются в файлы Claude
Незакрытые разговоры	Транскрипты и передача сессии по чек-пойнту	Остаются у старого CLI

Отсюда и ощущение обрыва: человек переключил провайдера и обнаружил, что агент забыл его правила, перестал соблюдать запреты и не знает ни одной его команды. Причём обнаружил не сразу, а в работе — сегодня никто не говорит ему заранее, что именно не доехало.

2. Решение: канон вместо девяноста переводов

Прямой путь — научить панель переводить из формата А в формат В — не масштабируется: десять CLI дают девяносто направлений, и каждое новое добавляет ещё восемнадцать. Поэтому перевод идёт не между парами, а через общую структуру: **канон**.



Стоимость честная: двадцать половинок вместо девяноста переводов, и новый CLI стоит двух — импортера и эмиттера, — а не восемнадцати. Обратное направление получается бесплатно: «человек писал правила под Qwen, переходит на Kimi» — это тот же импортер Qwen и тот же эмиттер Kimi, отдельного кода для этой пары не существует.

Половина уже написана

У панели есть читатель и писатель каждого раздела у каждого CLI — на них держатся сегодняшние экраны «Инструкции», «МСР», «Права», «Хуки», «Скиллы». Импортёр и эмиттер не разбирают форматы заново: они только приводят уже прочитанное к канону и обратно.

3. Канон несёт смысл, а не байты

Если бы канон был архивом файлов, он бы ничего не решал: файл хука Claude на Gemini — просто чужой текст. Поэтому каждая запись канона описывает не содержимое, а поведение. Ключевое поле — `needs`: какие факты рантайма записи нужны, чтобы работать.

ОДНА ЗАПИСЬ КАНОНА: ХУК, ЗАПРЕЩАЮЩИЙ АГЕНТУ ТРОГАТЬ ЧУЖОЙ БЭКЕНД

```
{
  "kind": "hook",
  "id": "backend-readonly",
  "source": { "provider": "claude", "file": "~/.claude/settings.json" },
  "intent": "Не дать агенту редактировать файлы бэкенда без разрешения",
  "trigger": { "event": "PreToolUse", "matcher": "Edit|Write" },
  "blocking": true,
  "needs": ["tool_name", "tool_input", "cwd"],
  "action": { "type": "command", "path": "~/.claude/hooks/backend-guard.mjs", "timeoutMs": 60000 },
  "sideEffects": ["deny-with-message"]
}
```

Из этой записи следует всё остальное, без чьего-либо мнения. Ей нужны имя инструмента и его аргументы — значит, на CLI, который о вызовах инструментов наружу не сообщает, она не может работать нативно. Она блокирующая — значит, деградация до «наблюдать и записывать в журнал» обязана быть названа, а не выполнена молча. У неё есть `intent` — значит, в крайнем случае её можно превратить в инструкцию модели человеческими словами, и человек увидит, чем именно ему это заменили.

Откуда берётся `needs` у уже написанного хука: панель разбирает его скрипт и смотрит, к каким полям входной нагрузки он обращается. Это гипотеза, поэтому она показывается человеку и правится им. Гипотеза, показанная явно, лучше молчаливого «наверное, доедет».

4. Пять уровней верности

Вопрос «будет ли потеря качества» без уточнения не имеет ответа. С уточнением — имеет, и ответ выражается пятью уровнями. Уровень считается для каждой записи отдельно и показывается ДО применения переноса.

КОД	УРОВЕНЬ	ЧТО ПРОИСХОДИТ С ЗАПИСЬЮ	РАБОТАЕТ, ЕСЛИ CLI ЗАПУЩЕН ВНЕ ПАНЕЛИ
Н	Нативно	У цели есть тот же механизм — запись переезжает в её формат	да
Э	Эмуляция	Механизма нет — его отыгрывает панель вокруг своего запуска CLI	нет
П	Провод	Нужен контур или DLP-прокси в пути запроса — там видно вызовы инструментов	нет

	Текстом	Превращается в инструкцию модели: соблюдение вместо принуждения	да
	Невозможно	Названо вслух с причиной; никакой уровень не спасает	—

Два правила, которые делают уровни честными

Уровень записи считается по ХУДШЕМУ из её `needs` : хук, которому нужны и текст запроса, и имя инструмента, не станет «почти нативным» оттого, что половина фактов доступна. И понижение всегда идёт в сторону строгости: `deny` не имеет права стать `allow` , а `ask` у CLI, который не знает «спросить», становится `deny` , а не «разрешено».

5. Матрица: слой × провайдер

Колонки — цели переноса; источником может быть любой из них. Матрица не пишется рукой: она вычисляется из карты возможностей провайдеров и поля `needs` каждой записи, поэтому новый CLI получает свои уровни сам, а расхождение таблицы с кодом ловится проверкой.

слой	CODEX	GEMINI	QWEN	CONTINUE	GOOSE	KIMI	CURSOR	OPENCODE	AIDER
Инструкции и память	Н	Н	Н	Э	Н	Н	Н	Н	Н
Скиллы	Э	Э	Н	Э	Э	Н*	Э	Н*	Э
Слэш-команды	Э	Н	Н	Э	Э	Э	Э	Н	Э
Субагенты	Э	Э	Э	Э	Э	Э	×	Э	Э
Хуки: события сессии	Э	Э	Н	Э	Э	Н	Э	Э	Э
Хуки: события инструментов	П	П	Н	П	П	Н†	П	П	П
Права	Т/П	Н‡	Н	Н	Т/П	Н	Н‡	Н	×
МСП-серверы	Н	Н	Н	Н	Н	Н	Н	Н	×
Переменные окружения	Н	Н	Н	Н	Э	Э	Э	Э	Н
Секреты и ключи	Э	Э	Э	Э	Э	Э	Э	Э	Э
Группы и сценарии	Э	Э	Э	Э	Э	Э	Э	Э	Э
Контекст разговоров	Э	Э	Э	Э	Э	Э	×	Э	Э

* `kimi` и `opencode` сами читают каталог `~/.claude/skills` — для них перенос скиллов это ноль работы, достаточно не мешать. † у `kimi` блокировать умеют только три события из шестнадцати, остальные наблюдательные: блокирующий `PostToolUse` деградирует и обязан это сказать. ‡ `gemini` не принимает режим «без вопросов» записью в файл, у `cursor` нет состояния «спросить» — правило понижается до запрета. У `cursor` нет чата через панель, поэтому контекст и субагенты для него невозможны.

6. Транспиляция по слоям: было → стало

6.1. Инструкции и память

Самый простой слой и единственный, который переносится сегодня, — но и он не сводится к копии файла. У Claude в `CLAUDE.md` есть `@`-импорты соседних файлов и служебная разметка разделов, которую понимает

панель. Ни того, ни другого у чужого CLI нет.

БЫЛО — `~/ .CLAUDE/CLAUDE.MD`

```
## ПРАВИЛО: Отвечать по-русски
Ответы человеку — всегда русский язык.

## ПРАВИЛО: Бэкенд только на чтение
Правки бэкенда — по отдельному разрешению.

@~/ .claude/rules/git-writes.md

## Отключённые правила (AgentDeck)
## ПРАВИЛО: Черновой режим
...
```

СТАЛО — `~/ .CODEX/AGENTS.MD`

```
## Отвечать по-русски
Ответы человеку — всегда русский язык.

## Бэкенд только на чтение
Правки бэкенда — по отдельному разрешению.

## Запись в git
<содержимое git-writes.md, вставлено целиком>

<выключенное правило не поехало>
```

Три операции, которых нет в копировании: импорты раскрываются в плоский текст (у цели нет механизма импорта, и ссылка превратилась бы в мёртвую строку); служебный префикс «ПРАВИЛО:» снимается, потому что у цели его никто не читает; выключенные правила не едут — иначе человек получил бы на новом CLI то, что у себя намеренно выключил. У `cursor` тот же канон раскладывается иначе: не один файл, а каталог `.mdc` -файлов с заголовочным блоком, по файлу на правило.

6.2. Силлы

Формат каталога силлов у `qwen`, `kim` и `opencode` тот же самый: папка с `SKILL.md`, имя и описание в заголовочном блоке. Поэтому перенос — копия каталога, а для `kim` и `opencode` и того меньше: они читают каталог Claude напрямую.

У остальных механизма силлов нет вовсе, и здесь начинается интересное. Силл у Claude — это не текст, а приём: модель видит короткий список «что есть», а тело силла подгружается только когда он понадобился. Ровно это панель и отыгрывает:

1. в инструкции целевого CLI подкладывается КАТАЛОГ силлов — имя и одна строка описания;
2. модель называет силл в ответе или срабатывает его триггер;
3. панель дописывает ТЕЛО силла в следующий запрос — и работа идёт дальше.

Почему тело не едет сразу целиком

Промпт уходит чужому CLI отдельным элементом командной строки, а она обрывается примерно на 32 тысячах символов; панель режет промт на 24 тысячах. Два десятка силлов целиком туда не влезут — влезет только их каталог. Поэтому тела едут файлом или дописываются по надобности, и это не оптимизация, а единственный рабочий вариант.

6.3. Слэш-команды

У `gemini` и `qwen` команды — файлы TOML, у `opencode` — markdown плюс ключ в конфиге, у остальных шести формата нет. Перевод прямой; там, где формата нет, команду разворачивает панель до запуска — она и так собирает промт.

```
---
description: Ревью текущей ветки
---
Прочитай diff ветки и найди
ошибки по списку: ...
```

```
description = "Ревью текущей ветки"
prompt = ""
Прочитай diff ветки и найди
ошибки по списку: ...
""
```

6.4. Хуки — главная работа партии

Хук — это не строчка в конфиге, а три вещи сразу: регистрация на событии, скрипт и **форма входных данных**, которую скрипт разбирает. Переносить только первое — значит получить зарегистрированный хук, который падает на первой строке.

```
"hooks": {
  "PreToolUse": [{
    "matcher": "Edit|Write",
    "hooks": [{
      "type": "command",
      "command": "node ~/.claude/hooks/guard.mjs",
      "timeout": 60
    }]
  }]
}
```

```
"hooks": {
  "PreToolUse": [{
    "matcher": "Edit|Write",
    "hooks": [{
      "type": "command",
      "command": "node ~/.qwen/hooks/shim-guard.mjs",
      "timeout": 60000
    }]
  }]
}
```

Разница не только в единицах таймаута (у Claude секунды, у qwen миллисекунды, у kimi снова секунды — перепутать значит получить хук с выдержкой в минуту вместо минуты). Разница в том, что командой ставится не сам скрипт человека, а **переходник**: он приводит нагрузку хозяина к той форме, которую скрипт ожидает от Claude, и зовёт скрипт по его собственному пути, не копируя и не правя его.

ЧТО ДЕЛАЕТ ПЕРЕХОДНИК

вход от Qwen → shim-guard.mjs → { hook_event_name, tool_name, tool_input, cwd, ... } → guard.mjs

форма Claude, которую скрипт уже умеет

поля, которых у Qwen нет, НЕ выдумываются: их просто нет в объекте, и скрипт отличает «нет данных» от «данные пустые»

У codex, gemini, continue, goose, cursor и aider хуков нет вообще. Для них работает следующий раздел.

6.5. Права

Здесь у каждого CLI своя модель, и честный перевод — это перевод в строгую сторону с отчётом, а не попытка сделать вид, что модели совпадают.

ЦЕЛЬ	ЧТО У НЕЁ ЕСТЬ	КАК ПЕРЕВОДИТСЯ DENY: BASH(RM -RF:*)
qwen, continue, opencode	Списки разрешить / спросить / запретить	Правило переезжает в список запретов
kimi	Упорядоченный список правил «решение + образец»	Переезжает с сохранением ПОРЯДКА — у него порядок значим
cursor	Только разрешить / запретить, состояния «спросить» нет	Запрет переезжает; ask из других правил станет запретом

codex , goose	Один общий режим согласований, правил нет	Пер-правило выразить нечем: режим ставится не слабее исходного, само правило уходит в отчёт и — при включённом контуре — в брокер прав панели
aider	Ничего	 названо вслух

6.6. Переменные окружения и ключи

Это слой, где «перенести полностью» и «перенести безопасно» кажутся противоречащими. Они не противоречат, если развести имена и значения.

Имена и несекретные значения

Едут в файл целевого CLI там, где у него есть раздел окружения (codex , gemini , qwen , continue , aider).

Значения секретов

Не едут в файл никогда. Панель подставляет их в окружение процесса в момент запуска CLI — из своего зашифрованного хранилища. На диске чужого CLI их нет.

У goose , kimi , cursor и opencode раздела окружения нет вовсе — там таким способом едут и обычные переменные тоже. Записать ключ в чужой файл можно, но только отдельной галочкой и с предупреждением: у чужих CLI нет зашифрованного хранилища, значит, это будет открытый текст на диске. Приём уже отработан в панели на профиле своего эндпоинта.

Как это проверяется

Значение-маркер кладётся в хранилище, делается перенос, и проверка ищет маркер во всех записанных файлах, резервных копиях, состоянии панели и журналах. Он обязан не найтись нигде и найтись ровно в одном месте — в окружении запущенного процесса, которое поддельный CLI сбросил на диск сам.

7. Надзиратель рантайма: панель как оболочка чужого CLI

Ключевой факт, из которого растёт вся эмуляция: **чужой CLI запускает панель**. Не человек в терминале — панель, своим процессом, со своим окружением, своим рабочим каталогом и своим транскриптом. А тот, кто запускает, имеет полное право исполнять события вокруг запуска. Это не вмешательство в чужую программу, это обычная работа оболочки.

Жизненный цикл прогона у чужого CLI — и где панель отыгрывает события Claude

1. Открытие

SessionStart

2. Запрос человека

UserPromptSubmit

блокирует

3. Работа модели

PreToolUse

PostToolUse

только через провод

4. Ответ готов

Stop

5. Закрытие

SessionEnd

Скрипты исполняются те же самые, по своим путям, с той же нагрузкой на стандартном вводе и тем же договором: код выхода 2 блокирует, JSON на выводе добавляет контекст или сообщение человеку. Для скрипта происходящее неотличимо от работы у Claude — а он и не должен отличаться.

СОБЫТИЕ CLAUDE	ЧЕМ ОТЫГРЫВАЕТСЯ У ЧУЖОГО CLI	ОГОВОРКА
<code>SessionStart</code> , <code>SessionEnd</code>	Открытие и закрытие разговора в панели	Полное совпадение
<code>UserPromptSubmit</code>	Момент отправки запроса, до запуска процесса	Блокировка работает по-настоящему: процесс не стартует
<code>Stop</code> , <code>SubagentStop</code>	Конец прогона; конец прогона дочерней группы разделения	Полное совпадение
<code>Notification</code>	Уведомления панели (они и так есть — телефон, Telegram, вебхук)	Совпадение по смыслу, не по поводам
<code>PreCompact</code>	Собственный чек-пойнт передачи сессии панели	У чужого CLI нет сжатия контекста — это ближайший аналог, и он назван
<code>PreToolUse</code> , <code>PostToolUse</code>	Только провод (раздел 8)	Снаружи процесса вызовы не видны — это граница, а не недоделка

Побочный выигрыш

Калитка запросов панели сегодня работает только у Claude — потому что построена на его событии `UserPromptSubmit` . С надзирателем она начинает работать у всех десяти: у панели теперь есть это событие везде. То же и с группами, и со сценариями.

8. Провод: то, чего не видно снаружи процесса

Вызов инструмента рождается внутри CLI: модель просит прочитать файл, CLI читает и кладёт результат обратно. Снаружи процесса этого не видно — видно только, что процесс что-то делает с диском. Поэтому события инструментов нельзя отыграть оболочкой, и врать об этом не нужно.

Но есть место, где вызов виден: запрос к модели. Если трафик CLI идёт через контур панели или её DLP-прокси, вызов инструмента лежит в теле ответа модели — панель его уже разбирает. Значит, на нём можно исполнить хук и, при коде выхода 2, не пропустить вызов дальше, подменив его ответом об отказе с причиной.

Две вещи, за которые здесь уже заплачено живым прогоном

Отказ обязан выглядеть для модели как ответ инструмента с ошибкой, а не как обрыв: пара «вызов↔результат» не должна разрываться, иначе разговор ломается на следующем шаге. И исполняется только то, что размечено протоколом: вызов внутри блока кода или в процитированной документации исполняться не должен — 12.09 настоящий CLI записал файл из блока, который модель явно пометила как пример.

Когда контур выключен, уровень честно падает до  и так и написан в отчёте. Панель не делает вид, что запрет работает, если работать ему нечем.

9. Два режима владения: перенос и подписка

Разовый перенос

Человек выбирает, что и куда. Панель показывает дифф каждого файла цели и отчёт верности. Он применяет. Дальше файлы живут своей жизнью, панель в них не лезет.

когда: разовый переезд

Подписка (проекция)

Канон живёт в панели и считается истиной. Каждый подписанный CLI пересобирается при изменении канона — по слоям: «скиллы во все, права никуда» законно.

когда: работа в двух CLI параллельно

У подписки есть ровно одна опасная точка: человек правил файл целевого CLI руками. Панель сверяет отпечаток того, что записала в прошлый раз, с тем, что на диске, и при расхождении **не перезаписывает**, а показывает дифф и три исхода: взять правку человека в канон, перезаписать проекцией, отписать этот слой. Ни один автоматический путь не затирает чужую работу — это тот же принцип, по которому уже устроены «незнакомые ключи» в адаптерах панели.

10. Контекст незакрытой работы

Среда переехала, а разговоры остались у старого CLI — переход всё равно ощущается обрывом. Поэтому при переключении панель предлагает перенести незакрытые разговоры: из каждого собирается чек-пойнт — тем же механизмом, которым она уже передаёт сессию при переполнении контекста, — и у нового CLI открывается чат с этим чек-пойнтом и корневой задачей.

Транскрипты при этом не переносятся и не конвертируются: чужих форматов истории не существует, и выдумывать их панель не станет. Переезжает смысл работы, а не её протокол. Старый чат остаётся нетронутым — перенос создаёт новое, а не портит старое.

11. Честная граница: что работает без панели

Это самая важная страница документа. Перенос делится на две половины, и человек должен знать, в какой находится каждая его настройка.

Половина «в файлах»

Инструкции, скиллы у трёх CLI, команды у трёх, права, MCP-серверы, имена переменных, хуки у `qwen` и `kimi`.

Человек запустил CLI сам, в своём терминале, вообще без панели — всё это работает. Это его файлы, в его форматах.

Половина «в панели»

Хуки у шести CLI, скиллы у шести, команды у семи, субагенты, ключи, группы и сценарии, события инструментов.

Работает, пока CLI запускает панель. В чужом терминале этого нет — и панель обязана писать это рядом с каждой такой записью, а не в сноске.

Что не даст никто и никогда

- **Аналитика, стоимость и шаги агента у чужого CLI.** Они читаются из потока событий, который документирован только у Claude. Не «пока не сделано» — граница продукта.
- **Песочница.** Раздел построен на возможностях Claude Code.

- **MCP y aider**. В его справочнике опций механизма нет; выдумывать ключ запрещено собственным правилом панели.
- **Одинаковое поведение модели**. Правила и скиллы соблюдает модель, а модели разные. Панель переносит среду, а не ум: слабая модель проигнорирует инструкцию, которую сильная соблюдёт. Это верно и внутри одного CLI при смене модели.

12. Чем это доказывается

Проверка стоит ровно того пути, который она исполняет. Поэтому три этажа, и ни один не заменяет другой.

ЭТАЖ	ЧТО ИСПОЛНЯЕТСЯ	ЧТО ДОКАЗАНО, А ЧТО НЕТ
1. Формат	Канон → запись → импорт → канон' на временном доме, настоящими адаптерами	Файл валиден и ничего не потеряно молча. Не доказано: что CLI это прочитает
2. Запуск	Поддельные CLI на PATH, сбрасывающие свои аргументы, окружение и стандартный ввод	До процесса доехало ровно задуманное. Не доказано: что CLI так себя поведёт
3. Поведение	Настоящий CLI, заглушка вместо модели, пять проб: хук блокирует · скилл срабатывает · право отказывает · инструмент MCP виден · переменная дошла	Единственный этаж, отвечающий на вопрос «работает ли так же»

Где мы сейчас честно стоим

Ни одного из девяти чужих CLI на машине разработки не установлено. Их форматы собраны по документации и закрыты круговыми тестами — это первый этаж. Второй достраивается поддельными CLI. Третий по каждому провайдеру закрывается отдельным пунктом живой приёмки, и до тех пор в отчёте верности он помечен как непроверенный. Это не оговорка в конце документа, а строка на экране рядом с провайдером.

Итог

Полный перенос без потерь возможен для инструкций, MCP-серверов, скиллов, команд, прав, переменных и ключей — при условии, что запуск идёт через панель. Часть из этого работает и без неё: то, что легло в файлы целевого CLI его собственным форматом. События инструментов работают только через контур. Аналитика, песочница и одинаковость самой модели не переносятся никогда — и названы вслух, чтобы не обнаруживаться в работе.

Порядок работ, задачи с критериями приёмки и способами проверки — `TASKS-PORTABILITY.md` в корне репозитория. Начинать можно с любой волны: канон и паспорт среды ничего не пишут на диск и дают человеку первую пользу — увидеть свою среду целиком и узнать, что именно доедет.